

MASTER RESUME — NOT FOR SENDING

This is a reference document. Cut bullets per job description to build a one-page sub-resume.

Arindam Langer

+91-7006395984 | arindamlanger@gmail.com | linkedin.com/in/arindam-langer | github.com/Arindam-Langer

TECHNICAL SKILLS (FULL LIST)

Languages: Go, JavaScript, Python, C, SQL, Rust (basic)

Frameworks & Libraries: Node.js, Express.js, React, Flask, LangChain, LangChainGo, Bubble Tea, gRPC

Databases & Queues: PostgreSQL, pgvector, MongoDB, Redis, BullMQ, Chroma

AI & ML: Ollama, OpenAI API, Deepgram, Google Gemini, RAG Pipelines, Llama3 fine-tuning, NEAT (neuroevolution)

DevOps & Tools: Docker, Kubernetes, AWS S3, FFmpeg, Git, Linux, CI/CD

Auth & Security: JWT, bcrypt, UUID v5 anonymization, token rotation & reuse detection, RBAC

Systems/Networking (studying): HTTP/2, gRPC internals, TCP fundamentals

EXPERIENCE (ALL BULLET VARIANTS KEPT)

Software Development Intern

August 2025 – August 2026

OnClick Innovations

Mohali, India

- Built 4 production AI video generation pipelines (UGC, Veo, Template) orchestrating 5+ providers (Google Gemini, HeyGen, ElevenLabs, Google Veo) — cut ad production time from hours to under 10 minutes.
- *Variant (impact-first):* Reduced ad production turnaround significantly (hours to under 10 minutes) by building 4 production AI video pipelines integrating 5+ third-party AI providers.
- Architected an async job queue with BullMQ and Redis handling 50+ concurrent AI workloads with retry logic, real-time progress tracking, and per-job credit metering.
- *Variant (infra-first):* Designed a Redis/BullMQ-backed async job queue supporting 50+ concurrent AI workloads, with automatic retry, real-time progress streaming, and usage-based credit metering.
- Engineered a template management system with RBAC publishing, tag/category search, and B-roll upload, plus a questionnaire-driven creation flow — served as the primary UX surface for 100% of platform users.
- Delivered a Live Ambient Scribe proof-of-concept in 3 days: real-time Deepgram audio capture streamed to OpenAI for live SOAP note and differential-diagnosis generation.
- **Tech Stack:** Node.js, Express.js, React, Python, Flask, Redis, BullMQ, FFmpeg, Docker, OpenAI API, Deepgram, Google Gemini, HeyGen, AWS S3
- **Context notes:** Product is Boostit.io. Internship ended August 2026 (aligns with graduation). Use the “Live Ambient Scribe in 3 days” bullet when a role values speed/POC ability. Use the queue/infra bullet when a role is backend-infra-heavy. Use the pipelines bullet when a role is AI-product-facing.

Backend Developer Intern

March 2025 – August 2025

Resolyte

Jammu, India

- Architected a query engine microservice in Go and PostgreSQL from scratch, serving 3+ downstream services via type-safe gRPC APIs with dynamic SQL generation (Squirrel).
- *Variant (systems-first):* Built a high-performance Go/PostgreSQL query engine microservice from the ground up, exposing type-safe gRPC APIs consumed by 3+ downstream services in a distributed system.
- Containerized with Docker and deployed on Kubernetes with CI/CD, achieving zero-downtime rollouts across 2 environments.
- **Tech Stack:** Go, PostgreSQL, gRPC, Docker, Kubernetes, Squirrel, CI/CD
- **Context notes:** This is the strongest pure-backend/Go evidence in the whole history — lead with this for backend infra roles. Prior gRPC/HTTP2 hands-on experience here is worth naming explicitly in interviews.

PROJECTS (FULL DETAIL, ALL VARIANTS)

- Authn-service — Authentication Microservice** | *Go, PostgreSQL, Redis, Docker* [GitHub](#)
- Built a 6-endpoint auth microservice in Go: JWT access tokens (10-min TTL), 7-day rotating refresh tokens, bcrypt hashing, and UUID v5 phone anonymization — zero raw PII stored.
 - Implemented token rotation with reuse detection — replayed tokens trigger atomic revocation of all user sessions; Redis blocklist auto-expires revoked access tokens via TTL.
 - Layered a 5-deep middleware chain (recovery, logging, rate-limit at 100 req/min per IP, blocklist check, 1MB max-body) and deployed via multi-stage Docker build with a 25-conn pgxpool and graceful SIGTERM shutdown.
 - **Load-tested with hey (12-core machine, native Go binary + Postgres 17 + Redis 7):** sustained 11,329 req/sec on the core middleware path at 50 concurrency (p50 3.9ms, p99 14.1ms), and 4,557 req/sec at 200 concurrency (p99 71.5ms).
 - JWT verification (hot path — HMAC-SHA256 verify + Redis blocklist GET): 1,538 req/sec at 50 concurrency (p99 75.1ms), 975 req/sec at 200 concurrency (p99 385ms).
 - Signup (bcrypt hash + Postgres insert): 135 req/sec at 10 concurrency (p99 137.8ms) — intentionally bcrypt-bound, a security property not a bottleneck. Signin (bcrypt verify + JWT mint + refresh token insert): 133 req/sec, 100% success (p99 130.6ms).
 - **Context notes:** Self-directed build, done to counter over-reliance on AI agents and build genuine ownership/understanding. Strongest “I understand what I build” evidence — good for interview deep-dives on token rotation, middleware design, and load-test methodology. Should be pinned on GitHub with a professional repo description. **Interview framing:** 11K req/sec is the middleware-chain number (rate-limit, logging, recovery included); 1.5K req/sec token verification is the real production-hot-path number; bcrypt-bound numbers (130ish req/sec) should be explained as a deliberate security tradeoff, not a limitation.

- OllamaChat — Privacy-First RAG Engine with TUI** | *Go, pgvector, Ollama, Docker, Bubble Tea* [GitHub](#)
- Built a fully offline RAG pipeline in Go: PDF ingestion, overlap-aware chunking, 768-dim embeddings via Ollama, and cosine-similarity retrieval over a Dockerized pgvector store — zero external API calls.
 - Designed a Bubble Tea TUI (Elm architecture) with streaming chat, multi-line input, a file browser, and multi-turn memory (top-5 context injection), reducing hallucination on domain docs.
 - **Benchmarked the offline preprocessing path** (Go benchmarks, `-count=3`, AMD Ryzen 5 5600H, 12 threads): text chunking (800-char/120-overlap) at ~79–81 MB/s single-threaded, ~167 MB/s parallelized (~2× via goroutines) — 8 allocs/op end-to-end.
 - SHA-256 file hashing (change detection) sustained ~1.6–1.8 GB/s streaming via `io.Copy`, constant 33KB memory footprint regardless of file size (tested 100KB–10MB).
 - Identified PDF text extraction (not embedding) as the actual pipeline bottleneck via benchmarking — hashing and chunking are both effectively free by comparison.
 - **Context notes:** Written substantially with AI agent assistance; useful for AI-infra-facing roles (RAG, embeddings, vector DBs) but pair with Authn-service in interviews to show independent capability too. Benchmarking work itself (identifying bottlenecks, not just headline throughput) is good evidence of engineering rigor — worth mentioning explicitly.

- AI Assistant for Farmers** | *Python, Flask, LangChain, Ollama, Chroma* [GitHub](#)
- Built a multimodal AI chatbot for plant disease diagnosis from image and text inputs, fine-tuning Llama3 with RAG using LangChain and Chroma for agriculture-specific retrieval.
 - Integrated feedback loops and real-time data retrieval to iteratively improve response accuracy and usability.
 - **Context notes:** Good for roles wanting applied ML/RAG + real-world domain framing (social-impact angle: agriculture).

SIDE PROJECTS (LOWER PRIORITY, RANGE/CURIOSITY SIGNAL)

- http-server** — hand-built HTTP server | *Rust* [GitHub](#)
- Built a basic HTTP server from scratch in Rust to understand request parsing and server internals below the framework layer.
 - **Context notes:** Signals systems-level curiosity outside Go. Minor project; use only if a role explicitly values Rust or low-level systems interest.
- NEAT-flappybird** — neuroevolution AI | *Python* [GitHub](#)
- Trained an AI agent to play Flappy Bird using the NEAT (NeuroEvolution of Augmenting Topologies) algorithm via `neat-python`.

- **Context notes:** Fun/ML-curiosity signal, not backend-relevant. Good for casual portfolio browsing, not resume bullets on backend-focused applications.

2048-game & asteroids — game logic | *Python*

[GitHub](#)

- Built classic game logic/rendering projects (2048, Asteroids) in Python.
- **Context notes:** Early-stage learning projects. Omit from sub-resumes; keep only in master doc for completeness.

EDUCATION

Central University of Jammu

Nov. 2022 – Aug. 2026 (graduated)

BTech in Computer Science and Engineering, Final CGPA: 7.6/10

Jammu, India

OPEN ITEMS / TO FILL IN LATER

- DSA/competitive programming stats (LeetCode count, Codeforces rating) if any exist.
- Certifications (AWS, GCP, Go-specific) if any.
- Hackathons, college clubs, mentoring, open-source contributions outside own repos.
- Confirm exact GitHub repo descriptions are updated to professional language (esp. OllamaChat and Authn-service).
- Full remaining GitHub repo list (17 total repos, 6 confirmed above) – check for anything missed.